

College of Engineering and Technology

Web Engineering Department
Arab Academy for Science and Technology

PROJECT DOCUMENTATION

Web Engineering

Project Title:

EVENTIFY

Supervised By	Dr. / Amr Fahmy
Course	Web Engineering
Semester	8
Academic Year	2026

Table of Contents

1.Project team	1
1.1 Team Members	2
1.2 Sub-Group Distribution	3
Introduction& problem Statement.....	4
2.1 Background	4
2.2 Problem Statement	4
2.3 Motivation	4
Project Objectives	5
Technology Stack	6
System Architecture & Design	7
5.1 Background	7
5.2 Frontend Architecture	8
5.3 Backend Architecture	8
5.4 Database Schema	9
Project Steps & Phases	11
Group Task Distribution	12
Development Timeline	13
Full Project Documentation	14
9.1 Features & Functional Requirements	14
9.2 Non-Functional Requirements	14
9.3 API documentation	15
9.4 UI/UX Design Notes	17
9.5 Testing Plan	20
9.6 Deployment & Devops	20
Testing plan	20

1. Project Team

1.1 Team Members

Full Name	College ID	Role / Responsibility	Email
Omar Khaled salama	221006032	Member	khaledashta@gmail.com
Mohamed Tamer Taymour	221004807	Member	mohtam7504@gmail.com
Youssef Emad abdefatah	221006764	Team Leader	youssefbayoumi72@gmail.com

1.2 Sub-Group Distribution

Sub-Group	Member(s)	Module / Layer	Key Deliverable
Frontend	All Members	UI / UX, Pages, Components	Responsive web pages
Backend	All Members	API, Auth, Business Logic	REST API endpoints
Database	All Members	Schema, Queries, ORM	DB design & migrations
DevOps / QA	Youssef Emad	Deployment, Testing	CI/CD pipeline & test reports

2. Introduction & Problem Statement

2.1 Background

The events industry has grown significantly in recent years, covering everything from concerts and conferences to local community gatherings. Managing event ticketing traditionally relies on manual processes, third-party platforms with high fees, or fragmented tools that don't connect organizers and attendees in one place. Technologies like Node.js, MongoDB, and transactional email APIs have made it feasible for smaller teams to build full-featured booking platforms without enterprise infrastructure. However, most lightweight solutions lack critical features like secure digital ticketing, real-time seat tracking, and integrated payment workflows. Eventify was built to fill that gap — a self-hosted, full-stack platform that gives organizers complete control and attendees a smooth end-to-end experience.

2.2 Problem Statement

1. **No centralized platform for small-to-mid scale event management** — organizers are forced to juggle spreadsheets, social media, and manual ticketing with no unified system to list events, track seats, and manage bookings in one place.
2. **No automated digital ticketing** — existing lightweight solutions rarely provide instant ticket delivery with scannable QR codes, leaving venues to rely on name lists or printed tickets that are easily lost or forged.
3. **Cash payment workflows are untracked** — when attendees pay in person, there is no system to log pending payments, confirm them, or flag who still owes, creating gaps between bookings and actual revenue.
4. **No real-time admin visibility** — organizers have no live view of seat availability, revenue collected, or booking statuses, making it impossible to make informed decisions before or during an event.

2.3 Motivation

Solving this problem matters because there is a clear and underserved gap between expensive enterprise platforms like Eventbrite and the chaos of managing events manually through spreadsheets, WhatsApp groups, or cash at the door. Smaller organizers — student unions, local businesses, and community groups — need professional-grade tools but cannot afford the fees or complexity that come with large commercial solutions. At the same time, modern attendees expect a digital-first experience: instant confirmation, a scannable ticket on their phone, and a smooth entry process. Eventify addresses both sides of this equation by providing organizers with real-time control over seats, payments, and bookings, while giving attendees an automated and trustworthy ticketing experience — all built on accessible modern web technology that can be self-hosted without enterprise infrastructure.

3. Project Objectives

By the end of this project, the team will have achieved the following measurable goals:

- **Build a fully functional multi-page web application** — delivering a complete event booking platform with distinct pages for home, login, registration, profile management, and password reset, all served from a unified Node.js backend.
- **Implement secure user authentication and authorization** — including JWT-based session management, bcrypt password hashing, OTP email verification, role-based access control separating regular users from admins, and a secure token-hashed password reset flow.
- **Develop a RESTful API with a NoSQL database** — designing and implementing four route groups (users, events, bookings, admin) backed by a MongoDB database with well-structured Mongoose models and proper input validation.
- **Build an automated digital ticketing system** — generating unique JWT-signed QR codes for every booking, delivering them instantly by email via the Brevo API, and providing a public ticket verification page scannable at venue entrances.
- **Implement a real-time admin dashboard** — giving administrators live visibility into total users, events, bookings, collected revenue, pending cash payments, and low-seat warnings through a dedicated stats endpoint.
- **Deploy the application to a cloud hosting platform** — hosting the full-stack application on Railway with environment-based configuration, production-ready security headers via Helmet, rate limiting, and MongoDB sanitization to protect against common web vulnerabilities.

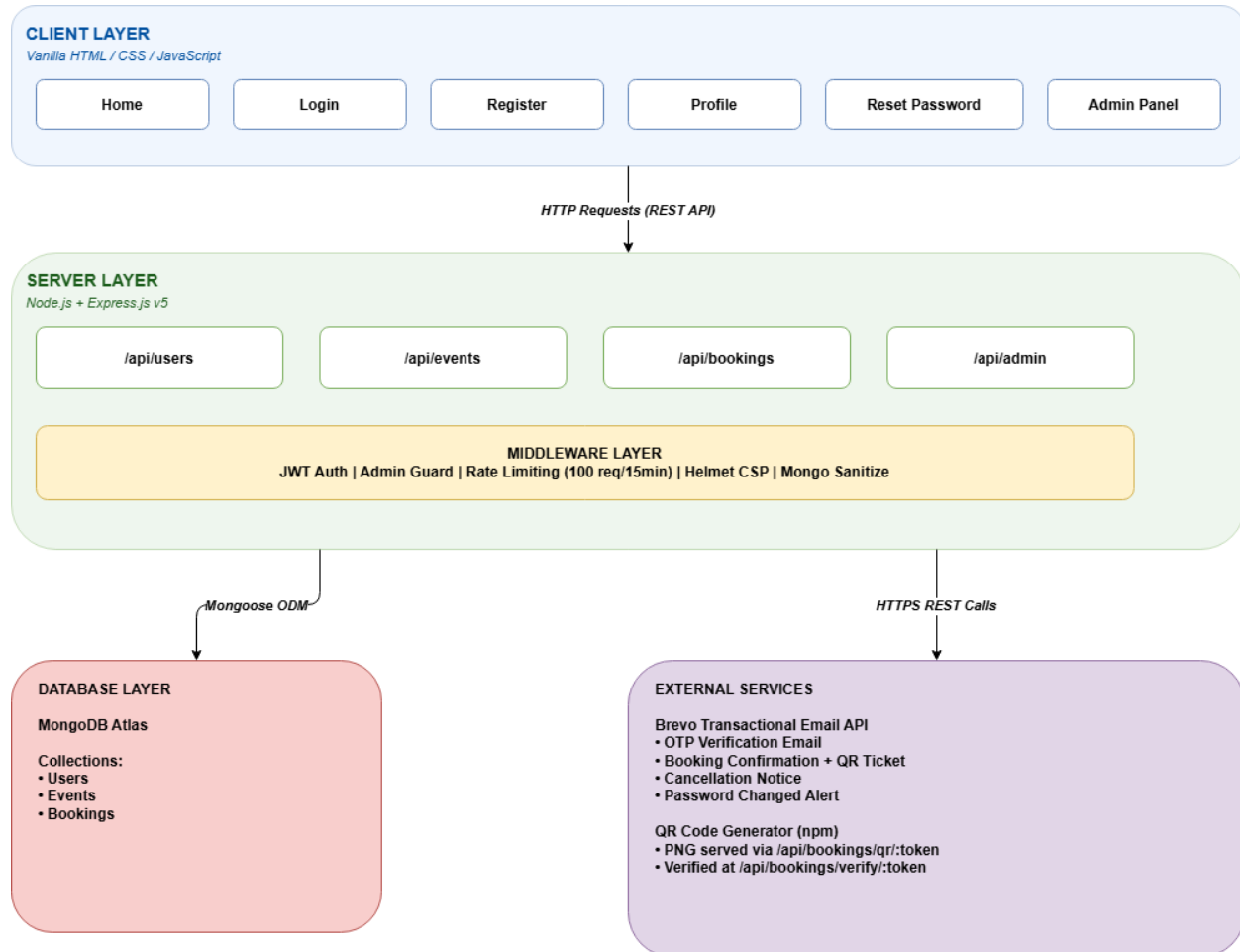
4. Technology Stack

Layer	Technology / Tool	Purpose / Notes
Frontend Framework	Vanilla html cs js	UI rendering & routing
Styling	Bootstrap	Component styling
Backend Framework	Node.js + Express	Server-side logic & APIs
Database	MongoDB	Data persistence
ORM / ODM	Mongoose	Database interaction layer
Authentication	JWT	User auth & sessions
Version Control	Git + GitHub	Source code management
Deployment	Railway	Hosting & CI/CD
Testing	Postman	Unit / integration / E2E tests
Email Gateway	Brevo	Email notifications/OTP

5. System Architecture & Design

5.1 High-Level Architecture

Eventify follows a classic client-server architecture with a clear separation between the frontend, backend, database, and external services. The diagram below illustrates how all major components interact:



The client layer is a set of static HTML/CSS/JS pages served directly by Express from the `/public` folder. Every user action (register, book, cancel) triggers an HTTP request to the server layer, which routes it through the appropriate Express router. Before reaching any protected route, requests pass through the middleware layer. JWT verification confirms identity, the admin guard checks permissions, rate limiting blocks abuse, and mongo-sanitize strips any injected operators from request data. The server then reads from or writes to MongoDB via Mongoose for all persistent data. On key events such as a new booking or account registration, the server calls the Brevo email API over HTTPS to deliver transactional emails. For bookings specifically, a JWT-signed QR code is generated and embedded in the confirmation email, linking to the public `/api/bookings/verify/:token` endpoint that staff can scan at the venue entrance to validate the ticket in real time.

5.2 Frontend Architecture

Pages

The frontend consists of five standalone HTML pages, each fully self-contained with its own HTML, CSS, and JavaScript files served statically by Express from the `/public` directory:

- **Home** (`/public/home/`) — main landing page displaying events with search and category filter functionality
- **Login** (`/public/login/`) — user login form with JWT token handling
- **Register** (`/public/register/`) — registration form with OTP verification step
- **Profile** (`/public/profile/`) — displays user info, booking history, and favorites
- **Reset Password** (`/public/reset-password/`) — token-based password reset form

Routing Strategy

There is no client-side router. Each page is a separate `.html` file navigated to via standard browser href links. Express serves all static files from `/public` and defines one explicit server-side route (`GET /`) that points to `home.html`. All other navigation happens through direct URL changes in the browser.

State Management

There is no state management library. State is handled through three mechanisms:

- **localStorage** — stores the JWT token and basic user info after login, making them available across pages without re-authenticating
- **In-page JavaScript variables** — each page's `.js` file manages its own UI state (form inputs, loaded data, loading flags) using plain `let` and `const` variables
- **API calls on page load** — each page fetches fresh data from the backend on every visit using `fetch()` with the stored JWT token in the Authorization header

Key Architectural Notes

- Every page is completely independent — there are no shared components, no imports between pages, and no build step or bundler involved
- Styling is also per-page with no shared CSS framework beyond Bootstrap loaded via CDN
- Authentication state is checked on each page load by reading the token from `localStorage` and redirecting to login if absent
- The admin panel is not a separate page but rather sections within existing pages that are conditionally shown or hidden based on the `isAdmin` flag decoded from the JWT token

5.3 Backend Architecture

API Design

The backend follows a RESTful API design with resource-based endpoint naming, all grouped under the `/api/` prefix. There are four resource groups: `/api/users` handles everything related to registration, login, OTP verification, profile management, favorites, and password changes. `/api/events` covers browsing, filtering, pagination, and full event CRUD. `/api/bookings` manages ticket creation, cancellation, QR code

generation, and public ticket verification. `/api/admin` exposes a single stats endpoint aggregating platform-wide data for the dashboard. Standard HTTP verbs are used consistently — GET to fetch, POST to create, PUT to update, and DELETE to remove.

Server Structure

The project follows a Routes + Models pattern, which is a lightweight variant of MVC. The entry point is `server.js`, which is responsible for connecting to the database, registering all middleware, mounting the four route groups, and starting the HTTP server. The `database/` folder holds the MongoDB connection logic. The `models/` folder contains the three Mongoose schemas — User, Event, and Booking — each defining the data shape, validation rules, and any pre-save hooks like password hashing. The `middleware/` folder contains two focused pieces of middleware: JWT verification and admin role checking. The `routes/` folder contains one file per resource, and each route file acts as both the router and the request handler, keeping the structure flat and easy to navigate without a separate controllers layer.

Middleware Stack

Middleware is applied in a deliberate order, from global to route-specific. At the global level, every incoming request passes through JSON body parsing with an 8mb limit to support base64 image uploads, MongoDB operator sanitization to strip injection attempts, Helmet for secure HTTP headers and Content Security Policy enforcement, and rate limiting capped at 100 requests per IP every 15 minutes. Static file serving for the frontend is also registered globally. At the route level, two middleware functions are applied selectively. The auth middleware verifies the Bearer JWT token on every protected route, decodes the payload, and attaches the user identity to the request object. The admin middleware runs after auth on admin-only routes and simply checks the `isAdmin` flag, returning a 403 if the user is not an administrator.

Key Design Decisions

Utility functions such as email sending, OTP generation, and token creation are defined inline within the route files that use them rather than in a shared utilities folder, keeping the codebase simple for the current project scale. Atomic seat booking is handled using a conditional MongoDB update that checks seat availability and decrements in a single operation, preventing race conditions under concurrent requests. Password reset tokens are SHA-256 hashed before being stored in the database, ensuring that a database leak cannot be used to hijack user accounts.

5.4 Database Schema

Entities

Users — represents a registered person on the platform. Key fields are `_id` (primary key), `email` (unique), `password` (bcrypt hashed), `isAdmin` (role flag), `isVerified` (email confirmation flag), `otpHash` / `otpExpires` / `otpAttempts` (OTP verification system), `resetPasswordToken` / `resetPasswordExpires` (password reset flow), and `favoriteEvents` (array of references to Event documents).

Events — represents an event created by an admin. Key fields are `_id`, `name`, `category`, `venue`, `locationUrl`, `image` (stored as base64 string), `startDate`, `status` (enum: `draft` / `upcoming` / `completed` / `cancelled`), `price`, `seats` (total capacity), and `available` (remaining seats, auto-decremented on booking).

Bookings — represents a ticket purchase linking a user to an event. Key fields are `_id`, `userId` (reference to User), `user` (email snapshot), `eventId` (reference to Event), `event` (name snapshot), `tickets` (quantity),

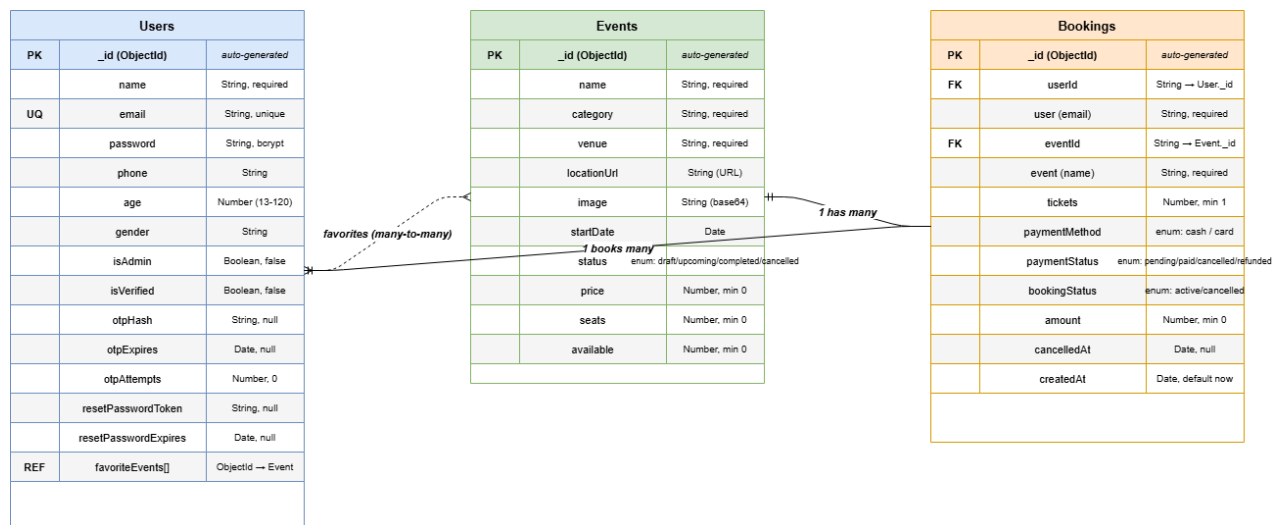
paymentMethod (cash or card), paymentStatus (enum: pending / paid / cancelled / refunded), bookingStatus (enum: active / cancelled), amount (price × tickets), cancelledAt, and createdAt.

Relationships

- User → Bookings (One-to-Many): one user can have multiple bookings, linked via userId on the Booking document.
- Event → Bookings (One-to-Many): one event can have multiple bookings, linked via eventId on the Booking document.
- User ↔ Events (Many-to-Many): a user can favorite multiple events, and an event can be favorited by multiple users, managed through the favoriteEvents array of ObjectId references stored on the User document.

Key Design Notes

- Bookings store both the foreign key (userId, eventId) and a snapshot of the name/email at the time of booking — this preserves booking history even if the user or event is later edited or deleted.
- The available field on Event is decremented atomically using MongoDB's findOneAndUpdate with a \$gte check to prevent race conditions when multiple users book simultaneously.
- The favoriteEvents array uses Mongoose populate() to resolve full Event documents when needed, keeping the reference lightweight in storage.



6. Project Steps & Phases

Phase	Step / Task	Expected Output / Deliverable
Phase 1	Project planning, requirements gathering, wireframes	SRS document, wireframes, approved scope
Phase 2	Database schema design, API contract definition	ERD, API documentation (Swagger / Postman collection)
Phase 3	Environment setup, project scaffolding, Git workflow	Initialized repo, CI/CD pipeline, branch strategy
Phase 4	Backend development — models, routes, controllers	Working API endpoints with Postman tests
Phase 5	Frontend development — pages, components, integration	Functional UI connected to backend API
Phase 6	Authentication, authorization, security hardening	Secure login, protected routes, role-based access
Phase 7	Testing — unit, integration, and end-to-end	Test reports, bug fixes
Phase 8	Deployment, performance optimization, final demo	Live URL, final report, presentation slides

7. Group Task Distribution

Module / Feature	Assigned Member(s)	Sub-Group	Status
UI Layout & Navigation	All Members	Frontend	Completed
Authentication Pages	All Members	Frontend	Completed
Dashboard / Main Feature	All Members	Frontend	Completed
Auth API (Register/Login)	Mohamed Tamer	Backend	Completed
Core Business Logic API	Omar Khaled	Backend	Completed
Database Design & Seeding	Youssef Emad	Database	Completed
Deployment & Hosting	Yossef Emad	DevOps	Completed
Testing & QA	Omar Khaled / Mohamed Tamer	QA	Completed

8. Development Timeline

Week	Phase / Task	Responsible Sub-Group	Deliverable
1	Kickoff — requirements, wireframes, planning	All	SRS + Wireframes
2	DB schema design & API contract	Backend + Database	ERD + API Docs
3	Environment setup, scaffolding, Git workflow	All	Running dev environment
4	Backend API development — Phase 1	Backend	Auth + core endpoints
5	Frontend development — Phase 1	Frontend	Core pages & components
6	Integration — frontend ↔ backend	All	Connected application
7	Testing, bug fixes, optimization	QA + All	Test reports
8	Deployment + final demo + report submission	DevOps + All	Live app + Final report

9. Full Project Documentation

9.1 Features & Functional Requirements

1. **User registration with OTP email verification** — new accounts require a 6-digit code sent to the user's email before access is granted, with a 10-minute expiry and a maximum of 5 attempts before a new code is required
2. **Secure login and session management** — JWT-based authentication with 7-day token expiry, covering both regular users and admins under the same login flow
3. **Password reset via email** — users can request a reset link sent to their email, backed by a SHA-256 hashed token with a 15-minute expiry
4. **Role-based access control** — two roles exist: regular user and admin, with the admin flag controlling access to event management, user management, booking overrides, and the stats dashboard
5. **Event browsing with search and filtering** — users can filter events by name, category, price range, date range, and status, with pagination support and sorting by name, price, or date
6. **Event management by admins** — admins can create, edit, and delete events including uploading a cover image as base64, setting venue, location URL, ticket price, total seats, and event status
7. **Ticket booking with seat control** — users can book one or more tickets for an event, with available seats atomically decremented to prevent overselling
8. **Dual payment method support** — users choose between cash (marked pending until confirmed) or card (automatically marked paid), with the booking status tracked throughout
9. **QR code ticket generation and delivery** — every confirmed booking triggers an email containing a styled HTML ticket with an embedded QR code image linking to the public ticket verification page
10. **Public ticket verification page** — a JWT-signed URL is generated per booking that renders a live ticket page showing holder details, event info, and booking status, scannable at venue entrance without requiring login
11. **Booking cancellation with seat restoration** — users can cancel bookings before the event starts, automatically restoring the seats to the event and triggering a cancellation email with refund information
12. **User profile management** — users can update their name, email, phone, age, and gender, with email changes automatically synced across related bookings
13. **Favorites system** — users can save and remove events from a personal favorites list stored on their account
14. **Admin dashboard statistics** — real-time aggregated stats including total users, total events, total bookings, active bookings, confirmed revenue, pending cash payments count, upcoming events count, and a low-seat warning list
15. **Transactional email notifications** — automated emails sent for OTP verification, booking confirmation, booking cancellation, and password changes

9.2 Non-Functional Requirements

1. **Performance** — API responses are kept lightweight by returning only necessary fields, using indexed queries on `userId` in bookings, and handling seat updates with a single atomic database operation rather than multiple round trips
2. **Security** — passwords are hashed with `bcrypt` before storage and never returned in any API response; JWT tokens are signed with a secret key and verified on every protected request; reset tokens and OTP codes are stored only as SHA-256 hashes; all request inputs are sanitized against MongoDB operator injection using `express-mongo-sanitize`; HTTP headers are hardened using `Helmet` with a strict Content Security Policy; IP-based rate limiting (100 requests per 15 minutes) protects against brute force attacks
3. **Scalability** — the API is fully stateless, with all session data carried in the JWT token rather than stored server-side, making it straightforward to run multiple instances behind a load balancer; MongoDB Atlas handles database scaling independently of the application server

4. **Reliability** — email sending is non-blocking and wrapped in try/catch so that a Brevo API failure never causes a booking request to fail; seat restoration on cancellation is handled transactionally to keep event availability consistent
5. **Responsiveness** — the frontend is built with Bootstrap loaded via CDN, providing a mobile-compatible layout across common screen sizes for all five pages
6. **Maintainability** — the codebase is organized into clearly separated folders for models, routes, middleware, and static frontend files, with each route file handling one resource, making it easy to locate and modify any feature independently

9.3 API Documentation

Users

Endpoint	Method	Description	Auth Required	Response
/api/users	POST	Register a new user	No	201 Created + email + requiresVerification flag
/api/users/login	POST	Login and receive JWT token	No	200 OK + token + user object
/api/users/verify-otp	POST	Verify email with 6-digit OTP code	No	200 OK + token + user object
/api/users/resend-otp	POST	Resend OTP verification code	No	200 OK + message
/api/users/forgot-password	POST	Request password reset email	No	200 OK + message
/api/users/reset-password	POST	Reset password using token from email	No	200 OK + message
/api/users/me	GET	Get current logged-in user profile	Yes	200 OK + user object + refreshed token
/api/users/me	PUT	Update current user profile	Yes	200 OK + updated user + refreshed token
/api/users/me/profile	GET	Get full profile for profile page	Yes	200 OK + full user document
/api/users/me/password	PUT	Change password (requires current password)	Yes	200 OK + message
/api/users/me/favorites	GET	Get user's favorite events	Yes	200 OK + array of events
/api/users/me/favorites/:eventId	POST	Add event to favorites	Yes	201 Created + updated user
/api/users/me/favorites/:eventId	DELETE	Remove event from favorites	Yes	200 OK + updated user
/api/users	GET	Get all users (admin only)	Yes (Admin)	200 OK + array of users

/api/users/:id	GET	Get user by ID	Yes	200 OK + user object
/api/users/:id	PUT	Update user by ID (admin only)	Yes (Admin)	200 OK + updated user
/api/users/:id	DELETE	Delete user and their bookings (admin only)	Yes (Admin)	200 OK + message

Events

Endpoint	Method	Description	Auth Required	Response
/api/events	GET	Get all events with optional filters and pagination	No	200 OK + array or paginated object
/api/events/:id	GET	Get single event by ID	No	200 OK + event object
/api/events	POST	Create new event with image (admin only)	Yes (Admin)	201 Created + event object
/api/events/:id	PUT	Update event details (admin only)	Yes (Admin)	200 OK + updated event
/api/events/:id	DELETE	Delete event (admin only)	Yes (Admin)	200 OK + message

Bookings

Endpoint	Method	Description	Auth Required	Response
/api/bookings	GET	Get bookings (own for users, all for admin)	Yes	200 OK + array of bookings
/api/bookings/:id	GET	Get single booking by ID	Yes	200 OK + booking object
/api/bookings	POST	Create new booking and send QR ticket email	Yes	201 Created + booking object
/api/bookings/:id/cancel	PUT	Cancel a booking and restore seats	Yes	200 OK + updated booking
/api/bookings/:id	PUT	Edit booking payment method (admin only)	Yes (Admin)	200 OK + updated booking
/api/bookings/:id/status	PUT	Update payment status (admin only)	Yes (Admin)	200 OK + updated booking
/api/bookings/:id	DELETE	Delete booking permanently (admin only)	Yes (Admin)	200 OK + message
/api/bookings/verify/:token	GET	Public ticket verification page (HTML)	No	200 OK + HTML ticket page
/api/bookings/qr/:token	GET	Serve QR code PNG image for ticket	No	200 OK + PNG image

Admin

Endpoint	Method	Description	Auth Required	Response
/api/admin/stats	GET	Get platform-wide dashboard statistics	Yes (Admin)	200 OK + stats object

9.4 UI/UX Design Notes

Design Language

Eventify follows a clean and modern dashboard-style design focused on simplicity and ease of navigation. The interface uses card-based layouts, clear visual hierarchy, and consistent spacing to provide an intuitive event browsing and booking experience. The design emphasizes accessibility and minimizes the number of steps required to complete common tasks such as event booking and payment.

Color Palette

- Primary Color: #0B6B3A (Eventify Green)
- Secondary Color: #DCEEE4 (Light Green)
- Background: #FFFFFF
- Text: #1F2937
- Success / Confirmation: #16A34A
- Warning / Attention: #F59E0B
- Danger / Logout & Delete Actions: #DC2626

Typography

- Font Family: Inter
- Base Font Size: 16px
- Headings: 32–48px, Bold
- Body Text: 14–16px, Regular
- Line Height: 1.5–1.6
- Button Labels: Medium Weight, 14–16px

Key User Flows

Guest User Flow

Home Page → Browse Events → View Event Details → Login/Register → Book Event

Authenticated User Flow

Login → Dashboard → Browse Events → Select Event → Choose Ticket Quantity → Select Payment Method → Confirm Booking

Cash Payment Flow

Select Event → Book Now → Cash Payment → Booking Confirmation → Visit Eventify Store → Payment Verification

Credit Card Payment Flow

Select Event → Book Now → Credit Card Payment → Enter Card Details → Confirm Payment → Booking Confirmed

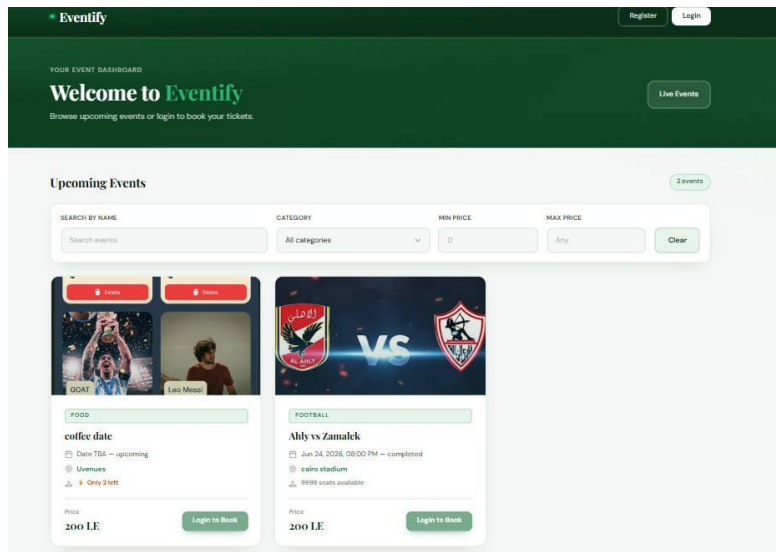
Admin Flow

Admin Login → Dashboard → View Statistics → Manage Events → Create/Edit/Delete Events → Monitor Bookings and Revenue

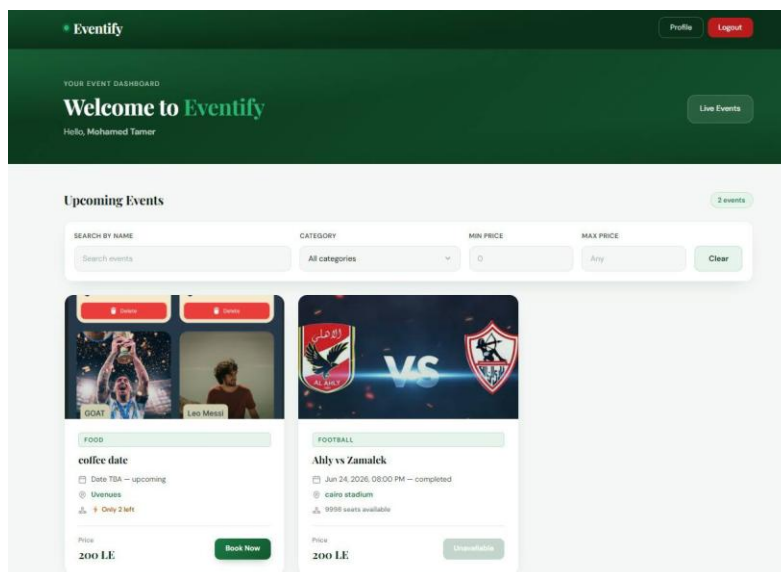
UI Mockups

The following screens were designed and implemented:

- Event browsing dashboard



- User event booking interface



- Admin dashboard and analytics page

ADMIN Admin Dashboard

REVENUE	USERS	EVENTS	BOOKINGS	CASH PENDING
211 EGP	21	5	19	11
Paid active bookings	Registered accounts	1 upcoming	13 active	Awaiting in-store payment

ATTENTION Low Seat Events 2

Event	Seats Left
coffee date	2 / 3 left
so7s makarem concert	4 / 30 left

Create New Event

EVENT NAME: CATEGORY:

VENUE: GOOGLE MAPS LINK:

EVENT PICTURE: No file chosen

TOTAL SEATS: PRICE (LE):

START DATE: STATUS:

Create Event

- Event creation and management form

Upcoming Events 2 events

SEARCH BY NAME: CATEGORY: MIN PRICE: MAX PRICE: **Clear**

GOAT

FOOD

coffee date

Date TBA — upcoming

Uvenues

Only 2 left

Price: 200 LE

Edit **Delete**

Ahly vs Zamalek

FOOTBALL

Ahly vs Zamalek

Jun 24, 2026, 08:00 PM — completed

cairo stadium

9998 seats available

Price: 200 LE

Edit **Delete**

- Payment selection and booking confirmation dialogs

Complete Booking

Choose your preferred payment method.

Event

se7s makarem concert

Price Per Ticket

100 LE

Total Amount

100 LE

NUMBER OF TICKETS

1

Select how many tickets you want to book

 **Cash Payment**
Pay at the nearest Eventify store

→

 **Credit Card**
Secure online payment

→

CARD HOLDER

John Doe

CARD NUMBER

4242 4242 4242 4242

EXPIRY

12/28

CVV

123

Confirm Payment

9.5 Testing Plan

Test Type	Tool / Method	Scope	Pass Criteria
Unit Testing	Jest	Functions, utils	> 80% coverage
Integration Testing	Postman	API endpoints	All endpoints pass
E2E Testing	Cypress	Critical user flows	All flows pass
Responsiveness	Chrome DevTools	Mobile & desktop	No layout breaks

9.6 Deployment & DevOps

Hosting Platform

Eventify is deployed on Railway as a cloud-hosted Node.js application. The backend API, static frontend files, and business logic are served from a single Railway service. MongoDB Atlas is used as the managed cloud database, while Brevo provides transactional email delivery for OTP verification, password reset, and booking notifications.

CI/CD Pipeline

Source code is managed using Git and GitHub. Development follows a branch-based workflow where features are developed independently and merged into the main branch after review and testing. Railway automatically redeploys the application whenever new changes are pushed to the production branch, providing a simple continuous deployment workflow.

Environment Variables

Sensitive configuration values are stored using environment variables and are never committed to the repository. Key variables include:

MONGODB_URI
JWT_SECRET
BREVO_API_KEY
EMAIL_FROM
CLIENT_URL
PORT

Separate configuration is maintained for local development and production environments using a .env file and Railway's secure environment variable manager.

Production URL

Frontend & Backend Hosting: Railway

Database: MongoDB Atlas

Production URL: <https://eventapp-production-bd3f.up.railway.app/>

10. Submission Checklist

- ✓ System architecture diagram completed
- ✓ Database ERD finalized and approved
- ✓ All API endpoints documented (Swagger / Postman collection)
- ✓ Frontend pages implemented and responsive
- ✓ Backend API tested and returning correct responses
- ✓ Authentication and authorization working correctly
- ✓ Frontend and backend fully integrated
- ✓ Database seeded with sample data
- ✓ Unit tests written and passing
- ✓ End-to-end tests covering critical flows
- ✓ Application deployed to production URL
- ✓ Environment variables secured (no secrets in repo)
- ✓ Code reviewed and merged to main branch
- ✓ Final project report completed and formatted
- ✓ Presentation slides prepared
- ✓ Live demo rehearsed by all team members